

APLICACIONES MOVILES

LABORATORIO 1

El taller y el primer proyecto

Módulo 2 — Desarrollo nativo · Guía autónoma · Tiempo estimado: 90–120 minutos (incluye instalación)

El problema de hoy: tenés una computadora y un teléfono. Al final de esta guía, tu teléfono va a estar corriendo una aplicación que escribiste vos — y vas a saber exactamente qué pasó en el medio.

Qué necesitás antes de empezar: una computadora con Windows 10/11 de 64 bits (las notas para Linux y Mac están al final de la Etapa 1), al menos 8 GB de RAM (16 recomendados), unos 15 GB de disco libres, conexión a internet estable (se descargan varios GB), y tu teléfono Android con su cable USB. No hace falta haber programado nunca en Kotlin: lo esencial se aprende acá, haciendo.

Teoría que acompaña: este laboratorio pone en práctica el ecosistema por dentro (las herramientas, la estructura de proyecto) y el recorrido del código a la app instalada. Si todavía no leíste esos bloques, podés hacer el lab igual — pero leerlos antes o después le va a poner nombre a todo lo que hagas acá.

El destino

Al terminar vas a tener: Android Studio funcionando en tu máquina; un proyecto llamado **HolaYo** — una carta de presentación digital con tu nombre, un dato tuyo y un botón que responde al toque—; esa aplicación corriendo primero en un teléfono emulado y después en tu teléfono real; y el archivo APK del paquete localizado en tu disco, con su peso anotado. Trivial como producto, completo como recorrido: vas a haber pasado por todas las estaciones que la teoría describe.

Etapa 1 — Armar el taller

La herramienta oficial del ecosistema es Android Studio: el editor, el depurador, los emuladores y la fábrica (Gradle), todo en un mismo lugar. Esta etapa lo deja instalado y listo.

1.1 — Verificá el terreno. Antes de descargar nada: abrí el Explorador de archivos → "Este equipo" y confirmá que tu disco principal tenga **al menos 15 GB libres**. Android Studio ocupa unos 3–4 GB, pero el SDK, los emuladores y los cachés de Gradle multiplican eso rápido.

✓ *Verificación: disco con 15+ GB libres. Si no los tenés, liberá espacio ahora — quedarse sin disco a mitad de la instalación es el atasco más frustrante de esta etapa.*

1.2 — Descargá Android Studio. Entrá a developer.android.com/studio (el sitio oficial — no descargues Android Studio de ningún otro lado) y bajá la versión estable para Windows. El archivo pesa alrededor de 1 GB [△ VERIFICAR: nombre y tamaño de la versión estable vigente al preparar el dictado].

✓ *Verificación: el instalador .exe descargado en tu carpeta de Descargas.*

1.3 — Instalá con las opciones por defecto. Ejecutá el instalador y aceptá las opciones que propone (siguiente → siguiente), incluida la instalación del "Android Virtual Device". No cambies la carpeta de destino salvo que sepas por qué.

✓ *Verificación: el instalador termina y ofrece "Start Android Studio".*

1.4 — El primer arranque: el asistente descarga el SDK. Al abrir por primera vez, Android Studio lanza su asistente de configuración. Elegí la instalación **Standard**, aceptá las licencias (botón "Accept" en cada pestaña — sí, hay que aceptarlas todas), y dejá que descargue: acá bajan el **SDK** (las bibliotecas y contratos de la plataforma contra los que se compila — el framework del que habla la teoría), las herramientas de compilación y una imagen de sistema para el emulador. Son varios GB: es el momento del mate.

✓ *Verificación: la pantalla de bienvenida de Android Studio ("Welcome to Android Studio"), sin avisos de error en rojo. Si llegaste acá, el taller está armado.*

 **Nota para Linux y Mac:** el recorrido es idéntico desde 1.2; cambia solo el formato del instalador (un `.tar.gz` que se descomprime en Linux — la página oficial trae las instrucciones exactas, incluidas las dependencias de 64 bits para distribuciones basadas en Debian—; un `.dmg` que se arrastra a Aplicaciones en Mac, eligiendo la versión correcta según tu chip sea Intel o Apple Silicon). Los puntos de verificación son los mismos.

Problemas frecuentes de la Etapa 1

- **La descarga del SDK falla o queda colgada** ("Unable to access Android SDK add-on list"): suele ser la red (proxy, WiFi institucional que bloquea). Probá con otra red; el botón "Cancel" en ese aviso específico del arranque no rompe nada — el asistente continúa.
- **Aviso sobre virtualización / "HAXM" / "Hyper-V"**: el emulador necesita la virtualización del procesador. Si aparece un aviso de que está deshabilitada, anotalo y seguí — lo vas a necesitar recién en la Etapa 3, y la solución (habilitar la virtualización en el BIOS/UEFI) está detallada en los Problemas frecuentes de esa etapa.
- **El antivirus hace todo lento**: algunos antivirus escanean cada archivo que Gradle toca (y son miles). Si tu máquina lo permite, agregá la carpeta del proyecto a las exclusiones del antivirus — la diferencia de velocidad es dramática.
- **"Tu carpeta de usuario tiene caracteres especiales"** (tildes, ñ, espacios): históricamente fuente de errores oscuros en las herramientas. Si tu usuario de Windows es, por ejemplo, "Ramón Pérez", y algo falla inexplicablemente en las próximas etapas, este es un sospechoso a recordar.

Etapa 2 — El primer proyecto, por dentro

Un proyecto Android no es una carpeta con código: es un contrato con la fábrica. Esta etapa lo crea y lo recorre por dentro, con la teoría de la estructura de proyecto en la mano.

2.1 — Creá el proyecto... con la plantilla correcta. En la pantalla de bienvenida, botón **"New Project"**. Android Studio abre la galería de plantillas — y acá está **el atasco clásico de este laboratorio**, así que leé antes de cliquear: la plantilla que vas a elegir es **"Empty Views Activity"** (fijate que diga *Views*). La que Android Studio destaca primero — "Empty Activity", a secas— crea un proyecto de Jetpack Compose, la forma moderna que vas a conocer en su momento; este laboratorio y los primeros trabajan sobre el **modelo tradicional**, que es el que te vas a encontrar en la enorme base de aplicaciones existentes — la teoría explica por qué se empieza por acá.

✓ *Verificación: la plantilla seleccionada dice exactamente "Empty Views Activity". Si tu proyecto ya creado tiene un archivo `MainActivity.kt` que menciona `setContent` y no existe ninguna*

carpeta `res/layout`, elegiste la plantilla equivocada: cerrá y volvé a crear — es más rápido que convertirlo.

2.2 — Configuralo. En la pantalla de configuración: **Name:** `HolaYo` · **Package name:** dejá el que propone (algo como `com.example.holayo` — el identificador único de tu app en el mundo; en un producto real sería tu dominio invertido) · **Language:** **Kotlin** · **Minimum SDK:** elegí **API 26 (Android 8.0)**. Pará un segundo en ese menú antes de seguir: al lado de cada versión, Android Studio te muestra el porcentaje del parque al que llegás — es, literalmente, la columna de cobertura de la tabla de versiones de la teoría, incrustada en la herramienta. Con API 26 llegás a más del 95% de los dispositivos del mundo [△ VERIFICAR: cifra que muestra la versión vigente de Android Studio]. Acabás de tomar tu primera decisión de alcance-vs-capacidad.

✓ *Verificación: los cinco campos como se indicó, botón "Finish".*

2.3 — La primera sincronización: la fábrica se enciende. Android Studio abre el proyecto y abajo aparece una barra de progreso que puede tardar varios minutos la primera vez: es **Gradle sincronizando** — leyendo la receta del proyecto, descargando las dependencias declaradas, preparando la fábrica. No toques nada hasta que termine.

✓ *Verificación: abajo a la derecha no hay barras de progreso activas, y no hay errores en rojo en la ventana "Build". Un aviso amarillo de actualizaciones disponibles es normal: ignoralo por hoy.*

2.4 — El recorrido: encontrá las tres cosas. A la izquierda, asegurate de que el panel del proyecto esté en vista **"Android"** (menú desplegable arriba del árbol de archivos). Ahora, el recorrido que la teoría prometió — encontrá y abrí cada una de las tres piezas:

- **El código:** `app` → `java` → `com.example.holayo` → `MainActivity.kt`. Una clase que hereda de `AppCompatActivity` — tu primera Activity: la pantalla. Fijate la línea `setContentView(R.layout.activity_main)`: la orden de "inflar" la estructura declarada en XML — el modelo tradicional en su gesto fundacional.
- **Los recursos:** `app` → `res`. Adentro: `layout/activity_main.xml` (la estructura de tu pantalla — abrílo: vas a ver el "Hello World!" declarado como un `TextView`), `values/strings.xml` (los textos — acordate de esta carpeta: acá vive después la internacionalización), `values/colors.xml`, `mipmap/` (los íconos).
- **El manifiesto:** `app` → `manifests` → `AndroidManifest.xml`. La declaración ante el sistema: fijate que tu `MainActivity` está registrada, y que tiene un `intent-filter` con `MAIN` + `LAUNCHER` — eso es lo que le dice al sistema "esta es la pantalla que se abre desde el ícono". No hay `main()` en ningún lado: hay componentes declarados, como dice la teoría.

Y una cuarta parada que no es de la aplicación sino de la fábrica: `Gradle Scripts` → `build.gradle.kts` (`Module :app`). Buscá adentro `minSdk` — ahí quedó escrita tu decisión del paso 2.2 — y el bloque `dependencies` — las bibliotecas que tu app usa.

✓ *Verificación (respondé mentalmente, sin mirar): ¿en qué archivo está declarado el texto "Hello World!"? ¿En qué archivo está registrada MainActivity ante el sistema? ¿En qué archivo vive el minSdk? — Si dudaste en alguna, volvé a recorrer: estas tres respuestas se usan todo el módulo.*

Problemas frecuentes de la Etapa 2

- **La sincronización de Gradle falla con error de red** ("Could not resolve..."): casi siempre es la conexión o un proxy. Verificá internet, y probá **File → Sync Project with Gradle Files** para reintentar.
- **"Gradle sync failed: Unsupported Java"** o similar: no instalaste otro Java ni configuraste nada — usá el que Android Studio trae embebido. Si tocaste la configuración de JDK, volvé a **Settings → Build Tools → Gradle → Gradle JDK** y elegí el "Embedded JDK".
- **Todo tarda una eternidad**: primera sincronización con descargas es lenta por naturaleza; las siguientes son rápidas. Si *siempre* es lento, revisá la exclusión del antivirus (Etapa 1) y cerrá otros programas pesados.

Etapa 3 — Correr en el emulador

Antes de tocar tu teléfono, la app corre en uno virtual: el emulador — la herramienta para probar sin comprar cien dispositivos.

3.1 — Creá tu dispositivo virtual. Abrí el **Device Manager** (ícono de teléfono en la barra lateral derecha, o menú **Tools → Device Manager**) → botón "+" / "Create Virtual Device". Elegí un teléfono de la lista (un **Pixel** mediano está bien — evitá por hoy los plegables y las tablets), y en la pantalla siguiente, la **imagen de sistema**: la versión de Android que el dispositivo virtual va a correr. Elegí una reciente que tenga el ícono de descarga y bajala (es un archivo grande — otro momento de mate). Nombralo como quieras y finalizá.

✓ *Verificación: tu dispositivo aparece listado en el Device Manager.*

3.2 — Ejecutá. Arriba, en la barra: verificá que el selector de dispositivo muestre tu emulador, y tocá el **botón verde de Play** (o Shift+F10). La primera vez el emulador tarda en arrancar (boteando un Android entero adentro de tu máquina — sé paciente: minutos, no segundos); después Android Studio compila, instala y lanza.

✓ *Verificación: el emulador en pantalla, con tu app abierta mostrando "Hello World!". Ese texto viajó desde `activity_main.xml`, pasó por la fábrica, se empaquetó, se instaló y se está ejecutando: acabás de recorrer el pipeline completo por primera vez — aunque todavía sin tocarlo. Eso viene ahora.*

Problemas frecuentes de la Etapa 3

- **El emulador no arranca y menciona virtualización / HAXM / WHPX**: la virtualización del procesador está deshabilitada. Hay que activarla en el BIOS/UEFI de tu máquina (se llama "Intel VT-x", "AMD-V" o "SVM Mode" según el fabricante; se entra al BIOS con una tecla al encender — buscá "habilitar virtualización + [marca de tu máquina]"). Es el problema serio más común de todo el laboratorio: si te trabás acá, es motivo válido de consulta en el foro *con el modelo de tu máquina y una captura del error*.
- **El emulador arranca pero queda en pantalla negra**: dale tiempo real (hasta 2-3 minutos la primera vez). Si persiste: en el Device Manager, menú del dispositivo → **"Cold Boot"**. Si aun así no: probá crear otro dispositivo con una imagen de sistema distinta.
- **"Waiting for target device to come online"** eterno: cerrá el emulador del todo y volvé a darle Play; si se repite, reiniciá Android Studio. (Sí: "apagar y prender" es una herramienta profesional real acá.)

- **La máquina se arrastra con el emulador abierto:** el emulador come RAM. Cerrá navegador con mil pestañas y todo lo pesado; si tu máquina tiene 8 GB, va a estar justa pero funciona. Y anticipo: en la Etapa 5 vas a correr en tu teléfono real — que además de honesto, es más liviano para tu máquina.

Etapa 4 — Hacerla propia: tu carta de presentación en Kotlin

Hasta acá, todo lo que corre lo escribió la plantilla. Ahora entra tu código — y con él, lo esencial de Kotlin: un modelo de datos, un valor que puede faltar, y una función que responde a un toque.

4.1 — Declarar la estructura (XML). Abrí `res/layout/activity_main.xml` y pasate a la vista "Code" (arriba a la derecha: Code / Split / Design). Reemplazá todo el contenido por esta estructura — una columna con dos textos y un botón:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvSaludo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvDato"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="16sp"
        android:layout_marginTop="8dp" />

    <Button
        android:id="@+id/btnSaludar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="@string/btn_saludar" />
</LinearLayout>
```

Fijate dos cosas mientras lo pegás: cada vista que el código va a tocar tiene un `android:id` (así se la encuentra después), y el texto del botón no está escrito acá — apunta a `@string/btn_saludar`, que todavía no existe. Por eso el editor lo marca en rojo: lo arreglamos ya.

✓ Verificación: la vista "Design" muestra la columna centrada; el único error pendiente es el string del botón.

4.2 — Los textos, donde corresponde. Abrí `res/values/strings.xml` y agregá adentro de `<resources>`:

```
<string name="btn_saludar">Saludar distinto</string>
```

Es un paso chico con una idea grande: **los textos fijos viven en recursos, no en el código ni en el layout** — la carpeta `values` puede tener después una variante por idioma, y tu app se traduce sin tocar una línea. La teoría lo retoma al hablar de internacionalización; vos ya lo estás haciendo bien desde el día uno.

✓ *Verificación: el rojo del layout desapareció.*

4.3 — El modelo y el comportamiento (Kotlin). Abrí `MainActivity.kt` y reemplazá la clase completa por esto — **cambiando los datos por los tuyos**:

```
package com.example.holayo

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

// Tu modelo de datos: una línea, y ya tiene igualdad, copia y representación.
data class Perfil(
    val nombre: String,
    val dato: String,
    val apodo: String? // el ? declara: "puede no haber apodo" — y el
    compilador lo vigila
)

class MainActivity : AppCompatActivity() {

    private val perfil = Perfil(
        nombre = "Ramón Estudiante",
        dato = "Estoy cursando Aplicaciones Móviles",
        apodo = null // probá también con un apodo real: "Rama"
    )

    private var saludoFormal = true

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvSaludo = findViewById<TextView>(R.id.tvSaludo)
        val tvDato = findViewById<TextView>(R.id.tvDato)
        val btnSaludar = findViewById<Button>(R.id.btnSaludar)

        // Si hay apodo se usa; si es null, el nombre. El operador ?: es el
        "plan B".
        val comoLlamarme = perfil.apodo ?: perfil.nombre

        tvSaludo.text = "Hola, soy $comoLlamarme"
        tvDato.text = perfil.dato

        // Una lambda: la función que se ejecuta cuando el botón se toca.
        btnSaludar.setOnClickListener {
            saludoFormal = !saludoFormal
            tvSaludo.text = if (saludoFormal)
                "Hola, soy $comoLlamarme"
            else
                "¡Buenas! Acá $comoLlamarme"
        }
    }
}
```

Leelo antes de correrlo, porque acá está el Kotlin esencial trabajando: la **data class** modela tu perfil en cuatro líneas; el **String?** declara que el apodo puede faltar y te obliga a decidir el plan B (el operador `?:`) — el error más famoso de la historia de Android, convertido en decisión de compilación; la **lambda** del `setOnClickListener` es comportamiento pasado como valor. Y el conjunto es el **modelo tradicional** en su gesto completo: estructura declarada en XML, código que la encuentra (`findViewById`) y la **muta** (`tvSaludo.text = ...`). Guardá ese gesto en la memoria: cuando conozcas la alternativa moderna, vas a entender exactamente qué cambió.

Ejecutá (Play).

✓ *Verificación: la app muestra TU nombre y TU dato; el botón alterna el saludo en cada toque. Probá también ponerle un apodo real y verificá que lo prefiera. Si algo no compila, el error más común es un id mal tipeado: compará cada `R.id.` con los ids del XML, letra por letra.*

Etapas 5 — Al teléfono

El emulador es conveniente; tu teléfono es la verdad. Esta etapa conecta ambos mundos.

5.1 — Despertá el modo desarrollador. En tu teléfono: **Ajustes** → **Acerca del teléfono** → buscá "**Número de compilación**" y tocalo **siete veces** seguidas (sí, siete; te va a pedir el PIN y anunciar "¡Ya sos desarrollador!"). Después: **Ajustes** → **Sistema** → **Opciones de desarrollador** → activá "**Depuración por USB**". (Los nombres exactos varían un poco por fabricante — si no encontrás algo, buscá "activar depuración USB + [marca de tu teléfono]").)

✓ *Verificación: el interruptor de Depuración por USB, encendido.*

5.2 — Conectá y autorizá. Enchufá el teléfono a la computadora con el cable. En la pantalla del teléfono va a aparecer un diálogo: "**¿Permitir depuración USB?**" con la huella de tu computadora — marcá "Permitir siempre desde esta computadora" y aceptá. Ese diálogo es el sistema haciendo exactamente lo que la teoría cuenta: nadie le instala software a un Android sin que su dueño lo autorice.

✓ *Verificación: tu teléfono aparece, con su nombre, en el selector de dispositivos de Android Studio (donde antes decía el emulador).*

5.3 — Ejecutá en tu teléfono. Con tu dispositivo seleccionado: **Play**. La app se compila, se instala y se abre — en tu mano.

✓ *Verificación: HolaYo corriendo en tu teléfono real. Momento de la foto.*

Y una reflexión de una línea antes de seguir, porque acaba de pasar algo que no viste: para instalarse en tu teléfono, el paquete tuvo que ir **firmado** — y vos no firmaste nada. Android Studio lo firmó por vos con una **clave de debug** que generó automáticamente: válida para desarrollar, inservible para publicar. La estación de la firma ocurrió, silenciosa; la firma de verdad —la identidad criptográfica de un producto— llega con el trabajo práctico.

Problemas frecuentes de la Etapa 5

- **El teléfono no aparece en el selector:** (1) probá otro cable — hay cables que solo cargan y no pasan datos, es comunísimo; (2) en el teléfono, al conectar, elegí el modo "Transferencia de archivos" si aparece la opción; (3) en Windows puede faltar el driver del fabricante: buscá "[marca] USB driver" en el sitio oficial del fabricante, instalá y reconectá.
- **Aparece como "Unauthorized":** el diálogo de autorización quedó sin aceptar (o lo negaste). Desconectá, en Opciones de desarrollador tocá "Revocar autorizaciones de depuración USB", reconectá y aceptá el diálogo.
- **"Installation did not succeed" / la instalación falla:** si alguna vez instalaste una app con el mismo nombre de paquete, desinstalala del teléfono y reintentá.

Etapla 6 — Encontrar el paquete

Todo lo que corrió hasta acá viajó empaquetado. Esta etapa lo pone en tus manos.

6.1 — Generá el APK. Menú **Build** → **Build App Bundle(s) / APK(s)** → **Build APK(s)**. Cuando termine, abajo a la derecha aparece la notificación "APK(s) generated..." con un enlace "**locate**": tocalo y se abre la carpeta.

✓ *Verificación: estás viendo el archivo `app-debug.apk`, típicamente en `app/build/outputs/apk/debug/`.*

6.2 — Pesalo y anotá. Click derecho → Propiedades. Anotá la ruta completa y el peso en el checklist de salida. Y la pregunta incómoda que vale la pena hacerse: tu app tiene dos textos y un botón — **¿por qué pesa varios megabytes?** Respuesta corta: adentro viajan las bibliotecas de compatibilidad, los recursos en todas las variantes, el empaquetado universal. La teoría contó la respuesta larga — y contó también el formato que la industria inventó para no mandarle a cada teléfono lo que no necesita. Tu APK de hoy es el punto de partida de esa conversación.

✓ *Verificación: ruta y peso anotados.*

Desafíos opcionales

Sin guía paso a paso — de eso se trata.

1. **El ícono propio.** Ese ícono genérico no te representa. Investigá la herramienta **Image Asset** de Android Studio (click derecho sobre `res` → New → Image Asset) y ponelo a HolaYo un ícono tuyo. Pista: el resultado aparece en `mipmap` — en varias resoluciones. ¿Por qué en varias?
2. **El paquete viaja.** Mandale el `app-debug.apk` a alguien de confianza (o a tu otro teléfono) por el medio que quieras e intentá instalarlo allá. Vas a encontrarte con advertencias: "orígenes desconocidos", análisis de Play Protect, un sistema que desconfía visiblemente. No las esquives: **leelas** — y después respondete: ¿de qué desconfía exactamente el sistema, si el paquete es tuyo? ¿Qué sabe la tienda de una app que este APK no puede demostrar? Acabás de tocar con las manos el modelo de confianza del paradigma: la firma de debug no acredita a nadie, y el peaje de la revisión existe por algo.
3. **El perfil crece.** Agregale a tu `Perfil` un tercer dato opcional (`String?`) — tu comida favorita, tu equipo — y mostralo solo si existe. Pista: investigá el patrón `?.let { }`.

Checklist de salida

- ☐ Android Studio instalado y operativo
- ☐ Proyecto HolaYo creado con la plantilla **Empty Views Activity**, minSdk 26
- ☐ Sé señalar sin dudar: dónde vive el código, dónde los recursos, dónde el manifiesto, dónde el minSdk
- ☐ La app corre en el emulador **y** en mi teléfono, con mis datos y el botón respondiendo
- ☐ Tengo la ruta y el peso del `app-debug.apk` anotados
- ☐ (Opcional) Ícono propio · APK instalado en otro dispositivo · tercer dato opcional

Qué de esto reaparece: el proyecto y su estructura, en todos los laboratorios que siguen; la decisión del minSdk y la firma, en el trabajo práctico; el gesto "declarar en XML, encontrar y mutar en código", en el Laboratorio 3 — donde vas a construir la misma app de tres maneras y entender por qué la industria cambió de gesto.